GUIDE

Setup Guide

Provisioning a development box — one playbook, every tier

PREPARED FOR

Infrastructure Team

Provisioning & operations

Applies to	**Every hardware tier**
Mechanism	**Ansible — push from a laptop, or pull by the box**
OS	**Ubuntu Server LTS, bare metal**
Companion documents	**Hardware Specification · Developer Guide**

PT ArtiVisi Intermedia
Software engineering & technical training
info@artivisi.com · artivisi.com

17 August 2026

Scope

This guide covers everything between two hand-off points:

- **Before:** the Hardware Specification's runbooks are done, including **Installing the operating system** — Ubuntu Server on the SSD, hostname already equal to the inventory name, the operator's key on `root` (not just on the account the installer made), ethernet wired, MemTest86 passed.
- **After:** developers log in and follow the Developer Guide.

Everything in between is one Ansible playbook, `ansible/site.yml`, and it is the same playbook on a salvaged 8 GB desktop and a 128 GB workstation. The tier changes the per-user ceilings, the quota, and whether the registry mirror and the standing loops run here; nothing else. The box's own inventory entry says who has an account and declares the few hardware facts the runbooks call out. There is no per-tier procedure.

The guide is organised the way the playbook is: a section per role, each saying what the role configures, why it is that way, which inventory values it reads, and how to see whether it worked. Shell appears only where you would type it — to verify or to diagnose. Configuration is not reproduced here; the roles under `ansible/roles/` are the source of truth, and this guide explains them.

How a box is provisioned

Push and pull

The playbook runs two ways against the same inventory and the same roles:

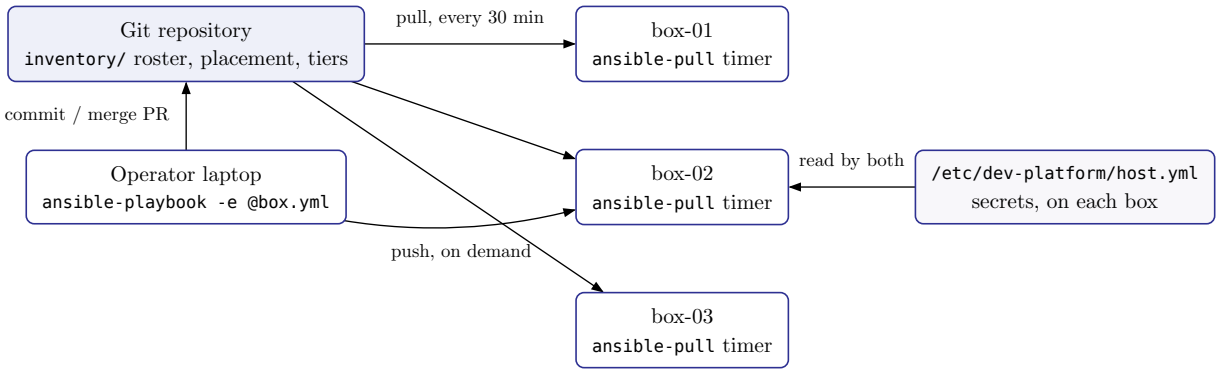

Figure 1: Push for the first run and for "now"; pull for everything after. Same playbook, same inventory, same result.

Pull is the normal mode. Every box runs `ansible-pull` on a systemd timer, clones `main`, and runs `site.yml` limited to its own hostname. A merged commit reaches every box within one interval; nobody runs anything. Drift — a hand-edited `authorized_keys`, a rebooted box — is corrected the same way. The `preflight` role gates every run, so a bad commit fails loudly on every box and changes nothing anywhere.

Push is for the first run on a new box (the timer does not exist yet), for "apply this now", and for the droplet test harness. Same command shape:

```
cd ansible
ansible-playbook -i inventory/hosts.ini -e @/path/to/<hostname>.yml site.yml
```

`-e @…` is the box's secrets file — the same content the box keeps at `/etc/dev-platform/host.yml` (next section). It never enters the repository.

What lives where

Item	Location	Notes
Roster — every person and their public keys	the deployment's `inventory/group_vars/all.yml` → `dev_people`	Public keys only. Self-service by pull request.
Placement — who is on this box	`inventory/host_vars/<hostname>.yml` → `dev_users`, `dev_users_absent`	Names only, from the roster. `dev_users_absent` deletes the account **and home**.
Hardware facts of this box	same file — `dev_lid_ignore`, `dev_mbpfan`, `dev_kernel_module_blacklist`, `dev_docker_data_root`	Only when the hardware runbook says so. Absent means "not this hardware".
Tier — capacity values	`inventory/hosts.ini` group + `group_vars/tier*.yml`	Exactly one tier group per host.
Platform constants — port blocks, paths, pnpm pin	`platform/ansible/group_vars/all.yml`	Set once by the platform; a deployment cannot override them.
Secrets & box-local: WireGuard client config, ntfy URL, E2E project list	**On the box:** `/etc/dev-platform/host.yml`, mode 0600	Never in git — every box holds a checkout. Template: `ansible/host.yml.example`.
Deploy key	**On the box:** `/etc/dev-platform/deploy_key`, mode 0600	Read-only key on the repository, one per box.

Nothing is defaulted. `preflight` asserts every required value and stops the run before anything is touched; a value missing from the wrong place is a clear message naming the file it belongs in.

Dry runs, and what they can and cannot see

`--check` runs the playbook without changing anything:

```
ansible-playbook -i inventory/hosts.ini -e @<secrets>/box.yml \
    --limit <box> site.yml --check
```

What it sees. Everything driven by a module — packages, file contents, templates, accounts, systemd units. That is most of the playbook, and a clean `changed=0` from a converged box is a real signal.

What it is blind to. Anything driven by `command`, because Ansible cannot know what an arbitrary command would do and so skips it. Timezone, the netplan `optional` edit, quota enablement and `update-grub` fall in that group: a box that has drifted on any of them still reports `changed=0` in check mode. Verified by drifting a box deliberately and watching the dry run miss it.

So read `--check` as **"nothing surprising in the module-driven half"**, not as **"this run will do nothing"**. It is a guard against a bad commit, not a proof of convergence. The proof is a real run reporting `changed=0`.

> **`--check` used to fail outright,** which is worth knowing if you meet an old copy of this repository. Read-only `command` tasks were skipped in check mode, and later tasks that parse their output then

failed on undefined variables — so a dry run ended in an error about `no element 0` rather than a report. The inspection tasks now carry `check_mode: false`, so they run and produce their output even in a dry run. They read state and change nothing, which is precisely why that is safe.

Bringing a new box up

Prerequisites, and the first command below fails on any of them: the hardware runbook and the OS install are finished, meaning `ssh root@<box>` works from your machine **without a password** and the box's hostname already equals the name it will carry in `hosts.ini`; a WireGuard peer for the box has been issued from the hub; you have the box's LAN address.

The key on `root` is the one that catches people. The installer puts it on the account it created, not on `root`, and this guide connects as `root` — see **Installing the operating system** in the Hardware Specification.

What a subiquity install leaves behind, found commissioning the first box. Four of these cost real time before anyone thought to look:

- **Package lists are cdrom-only.** The installer works offline, so `/var/lib/apt/lists` holds the ISO and `security.ubuntu.com` and nothing else. The first `apt install` of anything fails as "no package matching". `base` runs `update_cache`, so a full run fixes it; a tagged run that skips `base` does not.
- **A second network port with no cable stalls boot for two minutes.** subiquity writes every interface it saw into netplan, and netplan then requires each of them to come up before `systemd-networkd-wait-online` finishes. The empty one never does, so the unit burns its 120 s timeout with `cloud-init-network` blocked behind it. Mark it `optional: true`.
- **kdump reserves memory you were counting on** — 512 MB on a 16 GB box, taken before the OS sees it. Worth removing on a box you would reprovision from git rather than debug from a crash dump.
- **The installer allocates only part of the disk.** Guided LVM leaves the rest of the volume group unassigned — on the first box, 100 GB of 220 GB was in use and the remainder simply idle. `lvextend -r -l +100%FREE` while mounted.

1. **Add the box to the inventory** — a line under its tier group in `inventory/hosts.ini` (name = hostname, `ansible_host` = the box's address on the hub network: `hosts.ini` is also the address book developers read), and `host_vars/<hostname>.yml` with `dev_users`, `dev_users_absent`, `dev_pull_enabled: true`, plus any hardware facts the runbook called for. Commit.
2. **On the box, as root:** the secrets file and the deploy key.

```
install -d -m 700 /etc/dev-platform
# host.yml: WireGuard config, ntfy URL, E2E projects — see host.yml.example
vi /etc/dev-platform/host.yml && chmod 600 /etc/dev-platform/host.yml
ssh-keygen -t ed25519 -N '' -f /etc/dev-platform/deploy_key
cat /etc/dev-platform/deploy_key.pub   # add as a READ-ONLY deploy key on the repo
```

3. **First run, by push** from your laptop over the LAN address (WireGuard is not up yet — the playbook brings it up):

```
ansible-playbook -i inventory/hosts.ini -e @/tmp/<hostname>.yml site.yml
```

where `/tmp/<hostname>.yml` is a copy of the box's `host.yml`. Expect the full run to take some minutes on salvage hardware; every step is idempotent, so a run interrupted by anything can simply be repeated.

4. **Confirm the box now owns its convergence:**

```
ssh root@<hostname> systemctl list-timers dev-platform-pull.timer
ssh root@<hostname> systemctl start dev-platform-pull.service   # one run, now
ssh root@<hostname> journalctl -u dev-platform-pull.service -o cat | tail -3
```

   The recap must read `changed=0`. From here on, the laptop is optional.
5. Hand the developers the Developer Guide. Their accounts already exist; their keys are already in place.

Accounts

The `users`, `rootless_docker`, and `toolchain` roles together are user management. They are driven by two lists, and nothing else creates or removes accounts on a box.

The roster and placement

```
# inventory/group_vars/all.yml — this deployment
dev_people:
  alice:
    ssh_keys:
      - "ssh-ed25519 AAAA… alice@laptop"
      - "ssh-ed25519 AAAA… alice@desktop"
  bob:
    ssh_keys: ["ssh-ed25519 AAAA… bob@laptop"]

# inventory/host_vars/box-01.yml — this box
dev_users: [alice, bob]
dev_users_absent: []
```

`preflight` rejects: a placed name that is not in the roster; a name in both lists; a roster entry without a well-formed key; the name `e2e` (the loop service account). The same checks run in CI on every pull request (`platform/ansible/inventory-check.yml`), so a broken roster fails the PR rather than every box.

Changing someone's access

`scripts/access` performs each change as one command against this inventory, from a checkout on your machine. It edits the roster or the placement lists, commits, pushes, applies the `users` role to the boxes concerned, and moves the person's VPN peer — the parts that used to be a runbook, in the order that avoids the trap below.

Command	What it does
`grant <name> --key '<pub>' --box <box>`	Roster entry, placement, apply, mint the VPN peer, write out the `.conf` to hand over.
`revoke <name> [--key SUBSTR]`	**Suspend.** Empties their `authorized_keys` and withdraws the VPN peer. Account, home and every file untouched — this is a lost key, not a departure. `--key` drops one device when the others are still trusted.
`restore <name> --key '<pub>'`	Access back, with the replacement key.
`purge <name>`	**Removal.** Deletes the account and its home directory.
`prune <name>`	Housekeeping: drop them from the roster afterwards.

Why removal is two commands. `preflight` requires every name in `dev_users_absent` to still be in `dev_people`, so the roster entry has to outlive the removal — `purge` moves and applies, `prune` cleans up once the account is actually gone. Doing it in one step fails the run.

Why it runs from your machine and not on a box. A box holds a read-only deploy key, so a script there could not push the roster change, and without the push the next `ansible-pull` would recreate the account it had just deleted. Giving a box write access to fix that would let any box rewrite every box's configuration.

`scripts/test-access` checks the inventory editing after any change to the script: the failure it guards against is quiet, because a regex that eats a neighbouring key or a comment block corrupts the roster and Ansible applies the result without complaint.

Adding a person

The person opens a pull request adding themselves to `dev_people` — a name and one or more public keys. You review that the name is right and the key is theirs, merge, and add the name to `dev_users` of each box they should have. Within one pull interval they can log in. Nothing else: no ticket, no laptop run, no shell on the box.

Network reach is separate from the account and is not done by the playbook: the person's laptop needs its own peer on the WireGuard hub before their first login from outside the LAN. The playbook writes each **box's** client config from `host.yml` and stops there. Laptop peers are minted on the hub with `wg-add-peer` — the hub is this fleet's own, and `hub/README.md` documents it.

So a new developer needs two things from two places: a merged roster entry (above) and a peer (`sudo wg-add-peer <name> --person`). The Developer Guide tells them to ask for the second, and where to find the box's address (`ansible_host` in `hosts.ini`).

What the account comes with is fixed and the same on every box: a locked password (key-only), a private home (`0750`), lingering enabled, a rootless Docker daemon on its own socket, a memory ceiling and a disk quota wherever the tier enforces them, a 100-port block, and the version managers (SDKMAN, pnpm, uv) plus Claude Code — **no runtimes and no credentials**. What the person brings is in the Developer Guide.

Revoking a lost or stolen device

Developer keys carry no passphrase — a deliberate decision, recorded in the Developer Guide: a passphrase costs a daily keystroke and does not remove the need for revocation, it only adds a second route to it via forgotten passphrases. The trade is only sound if revocation is genuinely fast, so this is the procedure, and it is worth rehearsing before it is needed.

This is not offboarding. The person keeps their account and their files; one device's key stops working. Do not touch `dev_users_absent` — that deletes the home directory.

Access lives in three places and a key left in any one of them still works:

1. **The boxes.** Remove that key from the person's `ssh_keys` in `dev_people` (`group_vars/all.yml`), leaving their other devices' keys. `authorized_key` is `exclusive: true`, so the box drops it on the next run. Do not wait for the pull timer — push:

   ```
   ansible-playbook -i inventory/hosts.ini -e @<secrets>/box.yml \
       --tags users site.yml
   ```

2. **The VPN hub.** One command, on the hub. Other peers are unaffected:

   ```
   sudo wg-remove-peer <name>
   ```

3. **GitHub.** This one you cannot do for them. SSH keys belong to the person's own GitHub account, and no organisation admin can remove them — the person must, at github.com/settings/keys, from any device they still have. If they cannot get in at all, GitHub support is the path. Until that key is gone they can still push to any repository they could before.

Then have them generate a fresh key pair and go through Recipe 1, Steps 4 to 7 of the Developer Guide again.

From the person's side this is two actions, and that is all the Developer Guide asks of them: tell you, and delete the key from their own GitHub. The boxes and the VPN are yours to do; GitHub is theirs because it cannot be anyone else's.

> **The order matters if the device may be in hostile hands.** Do the VPN first: without it the boxes are unreachable from outside the campus, which buys time for the other two. GitHub is the one that stays exposed longest, because it depends on the person acting, so tell them to do it before you start.

Removing a person

Move the name from `dev_users` to `dev_users_absent`. On the next run the box: revokes the keys, disables lingering, terminates every process of the account (SSH, tmux, agent sessions, the daemon and every container), clears the quota entry and subordinate ranges, and deletes the account **and its home directory**.

> **Removal deletes data and nothing is backed up first.** `~/src`, `~/wt`, the image store, caches, credentials, and agent history are gone. Confirm with the developer that everything they want to keep is pushed before you merge. A name already absent is a clean no-op, so entries can stay in `dev_users_absent` as a record or be pruned later.

Renaming an account is not supported; it is a removal plus an addition. Moving someone between boxes is an addition on one and a removal on the other; no state moves with them.

Rotating or adding a key

Edit the roster. `authorized_keys` is written **exclusively** from it, so a removed key disappears and an added device appears on every box that lists the person, next interval.

What an account is, exactly

The per-user footprint is finite and known, and the removal path touches exactly this list: the passwd entry, `~`, `~/.ssh/authorized_keys`, one line each in `/etc/subuid` and `/etc/subgid`, the linger flag, the quota entry, and the user's own systemd units under `~/.config` (deleted with the home). Nothing box-wide is per-user — `/etc/profile.d/dev-platform.sh` is one file driven by UID, and it never carries a credential.

The roles

`site.yml` runs the roles below in this order. Every role is idempotent; a run with nothing to do reports `changed=0`, and the test harness asserts exactly that.

preflight

What. Asserts that every variable the run needs is defined, that the host is in exactly one tier group and its inventory name equals its hostname (pull limits itself by hostname), that the roster and placement are consistent, that secrets are present when the feature that needs them is on, that installed RAM meets the tier, and that the box is Ubuntu under systemd on cgroup v2. If quotas are on, that `/home` is on ext4. If pull is on, that the deploy key and `host.yml` exist with mode `0600`.

Why. The repository forbids fallback values. Without this role an unset variable would template as an empty string or take a module default, and the box would come up subtly wrong. Here it stops the play with a message naming the missing value and the file it belongs in.

Broken looks like: the run stops in the first seconds with a red assertion. That is the role working.

base

What. The apt archive mirror; timezone; the base packages (`git`, `tmux`, `btop`, `jq`, `ripgrep`, `smartmontools`, `acl`, and the rootless-Docker prerequisites `uidmap`, `dbus-user-session`, `slirp4netns`, `fuse-overlayfs`); `multi-user.target` so the box never boots a desktop; cgroup controller delegation to `user@.service`; the hardware-conditional settings — including `dev_network_optional_interfaces`, which stops boot waiting two minutes for an ethernet port with no cable in it, and `dev_kdump_enabled: false`, which returns the crash-dump kernel's memory at the next reboot; and quota accounting on the filesystem holding `/home` (`usrquota` in `fstab`, remount, `quotacheck`, `quotaon`).

Why delegation. On cgroup v2 the per-user memory ceilings the `users` role writes are accepted by systemd and then **silently ignored** unless the `cpu`, `memory`, and `pids` controllers are delegated to the user manager. A limit that is declared and not enforced is worse than none.

Why quota accounting here. `setquota` fails on a filesystem where quota accounting is off. Enabling it in the playbook rather than as a manual step means adding a user cannot fail on a box where the step was forgotten.

Why the mirror is pinned, first. `dev_apt_mirror` is rewritten into `sources.list.d/ubuntu.sources` before any package is installed, because every apt task after it depends on the archive answering. The country mirrors are the reason: `id.archive.ubuntu.com` and `sg.archive.ubuntu.com` both time out from here, and a mirror that works most of the time makes provisioning fail intermittently and at a task that looks unrelated. Only the archive line is touched — security updates come from `security.ubuntu.com` and stay there — and `Suites` and `Components` are left exactly as the installer wrote them, so nothing here needs editing at the next LTS.

Ansible's `replace` does nothing at all when its pattern misses, so the role reads the file back and asserts the mirror is present. Otherwise a box whose sources have an unexpected shape would sail past this task and fail later on a package error that says nothing about mirrors.

Verify.

```
cat /sys/fs/cgroup/user.slice/user-$(id -u alice).slice/user@$(id -u alice).service/
cgroup.controllers
# must list at least: cpu memory pids
quotaon -up /            # "user quota on / is on"  (or the /home mount)
grep URIs /etc/apt/sources.list.d/ubuntu.sources   # archive line == dev_apt_mirror
```

ssh_hardening

What. `PasswordAuthentication no`, `KbdInteractiveAuthentication no`, `PermitRootLogin prohibit-password`, in `sshd_config` and in every drop-in under `sshd_config.d` (cloud images ship one that re-enables passwords).

Why. One Unix account per developer, key-only, no shared logins. Shared accounts destroy the audit trail and make the per-user resource limits meaningless.

Why the drop-ins, specifically. Ubuntu carries `Include /etc/ssh/sshd_config.d/*.conf` near the top of `sshd_config`, and sshd keeps the **first** value it obtains for a keyword. A subiquity install ships `50-cloud-init.conf` holding `PasswordAuthentication yes`, which therefore beats anything written further down the main file. Hardening only the main file reads correctly in the diff and changes nothing in `sshd -T` — which is why the verify line below checks the effective configuration rather than the file.

Why validation runs once, after the drop-ins. A drop-in is a fragment; checking it alone proves nothing about what sshd actually reads, and Ansible's per-file `validate` cannot express "check the whole thing". The role writes every drop-in and then runs `sshd -t` over the combination, before the handler restarts anything. A bad edit costs a failed run, never access to the box.

Verify: `sshd -T | grep -i passwordauthentication` → `no`. Read `sshd -T`, not the file: it is the only thing that accounts for the drop-ins.

wireguard

What. Installs WireGuard, writes the client config from `dev_wireguard_config` (in `host.yml`), enables `wg-quick@wg0`. Refuses a config without `PersistentKeepalive`.

Why the keepalive is asserted. The box sits behind NAT. Without a keepalive the hub loses the return path when the tunnel goes quiet, and inbound SSH fails until the box next happens to send traffic. That is broken remote access, not degraded remote access.

> **Hub-side rule.** The hub runs plain `wg-quick` with no management UI, which is deliberate: a web interface that can mint peers is the keys to the whole network, and one that is internet-exposed is an open door. Peers are added over SSH with `wg-add-peer`, so root on the hub is the only thing that can issue them. Details in `hub/README.md`.
>
> When a laptop is on the same LAN as the box, SSH the LAN address directly — the VPN detour through the hub is for everywhere else.

Verify: `wg show` lists the hub peer with a recent handshake; SSH to the box's VPN address works from a laptop off the LAN.

users

What. Removes accounts in `dev_users_absent`; creates accounts in `dev_users` (locked password, home `0750`) and the `e2e` service account; writes `authorized_keys` from the roster; writes uid-derived subordinate UID/GID ranges; enables lingering and waits for each user manager; installs the per-user slice ceilings from the tier; installs `/etc/profile.d/dev-platform.sh` (`DOCKER_HOST`, `DEV_PORT_BASE`, `PNPM_HOME`, `PATH`); applies quotas; verifies lingering.

Why lingering is not optional. Without it systemd tears down the user's session scope when their last SSH connection closes — which stops their rootless `dockerd` and kills every running container with it. In this workflow that means: detach tmux, close the laptop, and the overnight agent run plus its whole stack dies silently, leaving a tmux session that looks alive on reattach but whose containers are gone. This is the single most common way a multiuser box appears to "randomly lose" long-running work.

Why slices, and why together with rootless Docker. Without limits, one developer's `mvn verify` consumes the box and everyone else's tests fail with timeouts that look like application bugs. The limit is set on the user's systemd slice; because rootless containers are children of the user's own `dockerd`, that one ceiling covers the user's **containers and JVMs together**. `MemoryHigh` throttles, `MemoryMax` kills — both on purpose: the box degrades visibly before it fails, then fails loudly rather than swapping itself into uselessness. The values come from the tier (table in the Hardware Specification), and each tier sizes them against the seat its workload actually holds rather than against a single fleet-wide figure — the tier's own section shows the arithmetic.

Why an empty `ssh_keys` list is legal. It means suspended: the account, the home directory and every file survive, `authorized_keys` is emptied, and since password authentication does not exist on these boxes the person cannot log in until a key is put back. That is the state to hold someone in while a replacement key is on its way, and it is deliberately not the same as removal, which deletes the home. `authorized_key` refuses an empty key, so the role truncates the file directly for such an account.

Why subordinate ranges derive from the UID. Rootless Docker maps container UIDs into the account's subordinate range. If ranges were allocated by list position, removing one user would shift every later user's range and orphan their image store. The UID is stable for the account's lifetime; the range is `100000 + uid × 65536`, and ranges cannot collide.

Why port blocks. Rootless Docker isolates container networks, but published ports still land in the host's single port space — two developers running the same service both want `:8080`. Each user gets 100 ports from `20000 + (uid − 1000) × 100`, exported as `DEV_PORT_BASE`; compose files reference it with `${DEV_PORT_BASE:?}` and fail if it is unset, rather than binding a colliding default.

Why quotas. One user filling the disk stops every other user's builds. With rootless Docker the image store lives under `$HOME`, so a home quota covers container images too — which is what makes runaway image pulls self-limiting. (On a two-disk box with `dev_docker_data_root`, the image store is on the data disk and outside the quota; the hardware specification budgets that disk for it.)

Verify.

```
loginctl show-user alice -p Linger --value           # yes
grep '^alice:' /etc/subuid                            # alice:<100000+uid*65536>:65536
systemctl show user-$(id -u alice).slice -p MemoryMax # the tier's value, in bytes
repquota -up /home | grep alice                       # soft/hard in KB
sudo -iu alice bash -lc 'echo $DEV_PORT_BASE'         # 20xxx
```

rootless_docker

What. Installs Docker Engine and masks the system daemon; empties the `docker` group; lifts `kernel.apparmor_restrict_unprivileged_userns` when the kernel has that knob; checks that an unprivileged process can create a user namespace; then per account: writes `~/.config/docker/daemon.json` (mirror, and `data-root` on the data disk when declared), runs `dockerd-rootless-setuptool.sh`, enables the user's `docker.service`, and verifies the socket answers.

Why rootless is mandatory. Membership in the `docker` group is equivalent to root on the host — any member can bind-mount `/` into a container and write anywhere as root. On a box where several developers hold projects under different client NDAs, a shared root daemon reduces that separation to convention. Rootless Docker moves each user's daemon into their own user namespace: `alice` cannot see, inspect, stop, or exec into `bob`'s containers, and there is no host path she can reach as root. Project isolation stops being a rule people follow and becomes a boundary the kernel enforces.

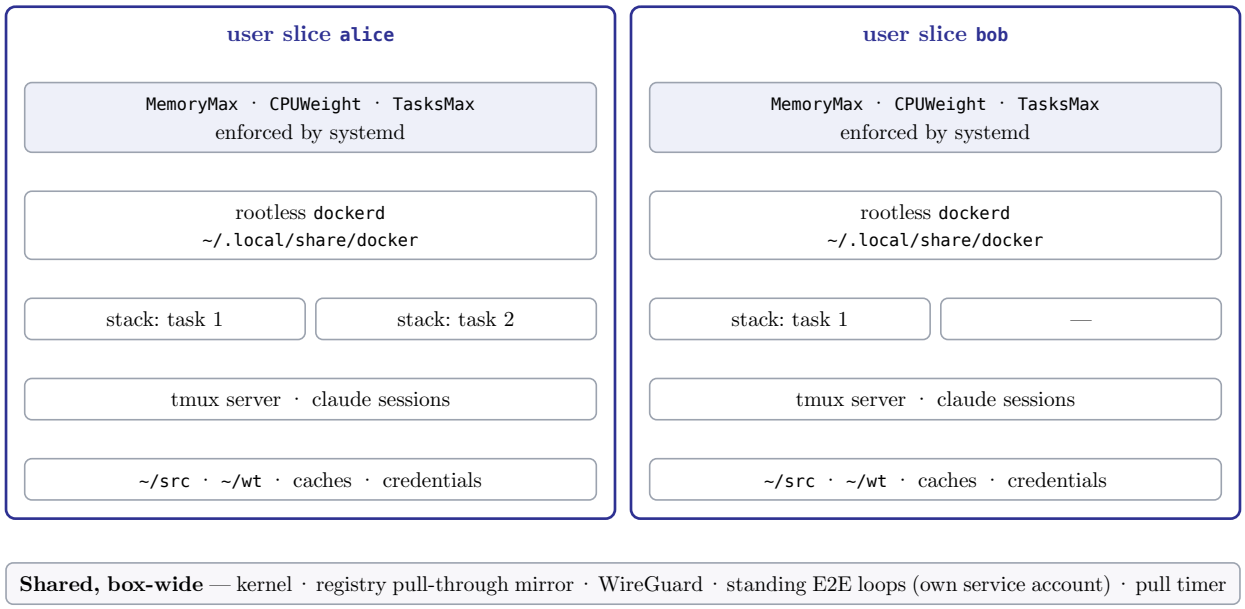

Figure 2: Two seats on one box — everything above the shared strip is namespaced per user and capped by that user's systemd slice

Why the AppArmor restriction is lifted. Ubuntu 24.04 introduced `kernel.apparmor_restrict_unprivileged_userns`, which transitions any unconfined process that creates a user namespace into the `unprivileged_userns` AppArmor profile. That profile denies the `/proc/self/exe` re-exec `rootlesskit` performs, so rootless Docker cannot start at all:

```
apparmor="DENIED" operation="exec" profile="unprivileged_userns"
name="/proc/self/exe" comm="rootlesskit" requested_mask="x"
```

The per-binary profile Docker's own installer suggests does not resolve this (verified on a clean box: the profile loads, `aa-status` reports `rootlesskit` unconfined, the transition still happens), and edits to the packaged profile are reverted by the next `apt` upgrade. The restriction is therefore lifted box-wide, only on kernels that have the knob — the playbook checks for it rather than for a release.

> **This is a real reduction in kernel attack surface, accepted deliberately.** Unprivileged user namespaces are the mechanism rootless Docker is built on, and rootless Docker is what replaces a root-equivalent shared daemon with one isolated daemon per developer. A box that cannot create unprivileged user namespaces cannot run this platform at all. The trade is a narrower privilege boundary between *users on this box* in exchange for a wider one between *unprivileged code and the kernel* — the right direction when the box's whole purpose is running several people's containers side by side.

Verify.

```
systemctl is-enabled docker.service           # masked
getent group docker                           # docker:x:NNN:   (no members)
sudo -iu alice docker info --format '{{.ServerVersion}}'
sudo -iu bob   docker ps -a                   # never shows alice's containers
```

registry_mirror

What. A pull-through cache of Docker Hub as a system unit under `podman`, listening on `127.0.0.1:5000`, data in `/srv/registry`. Every user's `daemon.json` points at it.

Why. Rootless Docker gives every user a **separate image store**, so without a mirror the same Postgres image is pulled once per user, every time it is refreshed. The mirror is the box-wide component that makes per-user isolation affordable: isolation costs disk, but not bandwidth. It runs as a system service, not inside any developer's rootless daemon — a mirror that dies when one user logs out is worse than no mirror — and under `podman` because the system Docker daemon is masked on these boxes and must stay that way.

Why host networking, and why both listeners are pinned. In proxy mode the registry contacts the upstream while starting and *panics* if it cannot resolve it — it does not retry. On podman's bridge network that lookup fails and the service crash-loops, logging a clean startup each time and no error of its own:

```
panic: Get "https://registry-1.docker.io/v2/": dial tcp: lookup
registry-1.docker.io ...: i/o timeout
```

Host networking uses the host resolver. It also means the container binds the host's interfaces directly, so both listeners are pinned to loopback explicitly — `REGISTRY_HTTP_DEBUG_ADDR` in particular, because registry 3 opens a debug server on `:5001` across all interfaces by default, and these boxes expose no inbound ports.

Verify: `curl -s http://127.0.0.1:5000/v2/` returns `{}`; `journalctl -u registry-mirror` shows no `panic`.

toolchain

What. Box-wide: `php-cli` and Composer from the distribution, `gh`, and one pinned `pnpm` binary at `/opt/pnpm`. Per user: the working directories (`~/src`, `~/wt`, `~/bin`, `~/.local/bin`, `PNPM_HOME`), SDKMAN, uv, Claude Code, and an initial `~/.tmux.conf`. **No runtime versions.**

Why managers, not runtimes. Which Java, Node, or Python a project needs is that project's business (`.sdkmanrc`, `.nvmrc` / `engines`, `.python-version`) and each developer installs it into their own `$HOME` through the manager on first login. A Laravel developer, a Node developer, a Java developer, and a devops user share a box without the box holding an opinion about any runtime version — nothing to go stale, nothing to collide. Version managers are per user for the same reason: one project's pin cannot disturb another user's build.

The PHP exception. There is no lightweight per-user PHP manager that does not compile from source, so `php-cli` and Composer come from the distribution, box-wide. Composer's per-user cache (`~/.cache/composer`) and per-project `vendor/` keep everything else per user.

Why pnpm is pinned and shared. It is the one binary the playbook downloads itself, so the version is explicit and bumped deliberately. The program is 146 MB and identical for everyone; what must stay per user — the Node version, global bins, and the content-addressable store — is `PNPM_HOME`, set per user in `profile.d`. The published `install.sh` is deliberately not used: it resolves the home directory from the invoking process rather than `$HOME` (so it fails under automation), rewrites the user's shell rc which `profile.d` already owns here, and always fetches the latest release.

Verify: `sudo -iu alice bash -lc 'sdk version; pnpm --version; uv --version; claude --version'`.

e2e_loops

What. The `e2e` service account (`/srv/e2e`, lingering, its own rootless daemon), the runner `/srv/e2e/bin/e2e-run.sh`, the template unit `e2e@.service` and timer `e2e@.timer` (every 15 min, randomised by up to 5 min), one timer enabled per name in `dev_e2e_projects`.

Why a service account. Standing loops are mechanical verification: they must not burn agent capacity, and they must not run as any developer's account, or they stop when that person's session ends and inherit whatever is in their environment.

Why the randomised delay. Pushes landing on two projects in the same window must not fire both suites at the same instant — a suite slowed by CPU contention produces timing flakes, which is the one thing a standing loop must never do.

What the box does and what the project does. The runner fetches `origin/main`, exits quietly if nothing moved, otherwise resets to it and runs the project's own `bin/e2e.sh`. Failure posts to `dev_ntfy_url`. A missing `bin/e2e.sh` is a hard failure by design — a project that silently never verifies is worse than one that fails loudly on the first run. Cloning each project into `/srv/e2e/src/<name>` is a one-time manual step as `e2e` (`sudo -iu e2e git clone …`); the repository access it needs is that project's business.

Verify: `systemctl list-timers 'e2e@*'`; after a push to a project's `main`, `journalctl -u e2e@<project>` shows the run.

> **The loop account counts as a seat.** Its stack occupies the same RAM as a developer's, and its build occupies the same cores. Size the box for *developers + standing loops*, not developers alone.

pull

What. `ansible-core` and `git`; the `ansible.posix` collection; `dev-platform-pull.service` (one `ansible-pull` run against `dev_repo_url`, branch `dev_repo_branch`, inventory from the checkout, extra vars from `host.yml`), `dev-platform-pull.timer` (`dev_pull_on_calendar` in `group_vars/all.yml` — every 30 minutes), and `dev-platform-pull-failed.service` as the `OnFailure` target, which posts to `dev_ntfy_url`.

Two guards worth knowing. `ansible-pull` exits 0 when the inventory has no host matching the box's hostname — a run that "succeeded" by doing nothing. The unit's `ExecStartPost` therefore asks the inventory for the hostname and fails the run if it is not there. And `--clean` discards local edits in the checkout, so nobody can quietly diverge a box by editing `/var/lib/dev-platform/repo`.

Why the deploy key is passed in a quoted `Environment=`. systemd splits an unquoted `Environment=` value on whitespace, so `Environment=GIT_SSH_COMMAND=ssh -i /etc/dev-platform/deploy_key -o ...` sets `GIT_SSH_COMMAND=ssh` and discards the key and every option as "invalid environment assignment". Git then authenticates with no identity and the pull dies on `Permission denied (publickey)` — which points at the deploy key, the one thing that is fine. The value is quoted for that reason. Note the droplet harness cannot catch this: it drives `ansible-pull` with a local `file://` URL, and the template only emits this line for a remote one.

Consequences to accept. The box needs outbound Git access, and the deploy key grants read of the whole private repository — which is why secrets are not in it. `main` is production: a merge is a deploy to every box. The CI check and the droplet harness are how a change is tested before it lands.

Verify: `systemctl list-timers dev-platform-pull.timer`; `journalctl -u dev-platform-pull.service -o cat | grep -E 'RECAP|changed='`.

Targeted runs and tags

A full run with no tags is the norm and is what the timer does. For a push run that should touch one area only:

Tag	Roles
`users`	users, rootless_docker, toolchain — accounts, keys, lingering, quotas, daemons, managers
`limits`	users — slice ceilings (with the rest of the role; it is cheap)

Tag	Roles
`mirror`	registry_mirror
`e2e`	e2e_loops
`pull`	pull
`base` · `ssh` · `wireguard`	the role of the same name

`preflight` always runs. Example — add a person to one box, now:

```
ansible-playbook -i inventory/hosts.ini -e @/tmp/box-01.yml site.yml \
  --limit box-01 --tags users
```

Troubleshooting

Symptom	What it is and where to look
A step hangs with no output — a build, an install, an agent	`MemoryHigh` throttling: the process crossed the ceiling and is in continuous reclaim, never erroring. `cat /proc/<pid>/wchan` reads `__mem_cgroup_handle_over_high`. The ceiling must clear the largest single tool's working set, not the steady state.
A build dies with no stack trace; a test suite reports failure with no cause	OOM kill at `MemoryMax`. `journalctl -k --since "10 min ago" | grep -i "killed process"`; `systemctl --user status docker` as the user for container deaths. Brief every user on this — a limit people can read is a capacity signal, one they cannot is a flaky test.
Overnight work "vanished"; tmux looks alive but containers are gone	Lingering is off for the account: `loginctl show-user <name> -p Linger`. The `users` role verifies it; if it drifted, a pull run restores it.
Memory limit written but `memory.max` reads `max`	Controllers not delegated — `base` writes the drop-in; a reboot may be needed for `user@.service` to pick it up.
Registry mirror crash-loops with clean startup logs	DNS from the container — the unit already uses host networking; check the host resolver.
Box unreachable over VPN after a quiet period	Missing `PersistentKeepalive` in the client config; `wg show` on the hub shows an old handshake.
Pull run fails, ntfy message received	`journalctl -u dev-platform-pull.service`. Most often a preflight assertion from a bad commit — fix in git; or the hostname is not in the inventory (`ExecStartPost` guard).
`setquota` fails: no quota enabled	`/home`'s filesystem was not remounted with `usrquota` — `base` does this; run it (`--tags base`) and check `quotaon -up`.

Testing a change

Every change to `ansible/` is tested against a throwaway DigitalOcean droplet before it lands on `main` — because landing on `main` is deploying to every box.

```
cd ansible
export DO_SSH_KEY_ID=<id from `doctl compute ssh-key list`>
export DEV_TEST_PUBKEY=$HOME/.ssh/id_ed25519.pub
./test/provision.sh && ./test/run.sh && ./test/destroy.sh
```

`run.sh` proves the properties this guide rests on, in order: push provisioning succeeds and `verify.yml` passes; a second run is `changed=0`; the droplet provisions **itself** by `ansible-pull` and that run is `changed=0`; moving one name to `dev_users_absent` and adding another removes the account cleanly, adds the other, and leaves a container the surviving user had running untouched; and the churn re-run is `changed=0`. It uses the smallest droplet that survives its own provisioning (`s-1vcpu-2gb`); it verifies mechanism, not capacity. Destroy the droplet when finished — nothing else reclaims it.